

Klausurenplan

Allgemeine Hinweise

- Alle zu bearbeitenden Klassen finden Sie im Paket `de.uniwue.jsp.klauplan`.
- Das Paket befindet sich im Ordner `src`.
- Die Skelette der Klassen sind bereits gegeben, die dort eingefügten Exceptions sind durch den erwarteten Code zu ersetzen.
- Es ist gestattet, aber nicht nötig, neben den geforderten Klassen weitere Klassen oder zusätzliche Methoden zu implementieren.

Unit-Tests

- Es stehen lokale Unit-Tests zum Überprüfen Ihrer Implementierung zur Verfügung.
- Die Testklassen befinden sich im Ordner `test`.
- Jede Klasse in dieser Aufgabenstellung hat eine passende Testklasse (Name: `"Test<Nr.><Klassenname>"`). Diese können auch einzeln ausgeführt werden.
- Die Tests sind von Anfang an ausführbar.
- Die Fehlermeldungen der Tests sind unter Umständen nicht sehr aussagekräftig. Sie haben aber die Möglichkeit die Tests zu **debuggen** und herauszufinden welche Szenarien getestet werden.

Aufgaben

Sie sollen ein System schreiben, das sich um die Verwaltung von geplanten Klausurterminen und zugehörigen Räumen kümmert. Dazu müssen von Ihnen Klassen zur Datenhaltung implementiert werden, sowie eine Analyse-Methode, um Informationen aus einer vorgegebenen Planung zu extrahieren.

Es sind Datenklassen für Räume, Prüfungen und Studenten anzulegen. Die tiefergehende Datenaufbereitung geschieht dann in der Analyse-Klasse.

Die Klausur ist in diese Teilaufgaben gegliedert:

Nummer	Name	Punkte
1.	Raum	10
2.	Pruefung	18
3.	Student	15
4.	Analyse	47
Summe		90

Das Erreichen von **60** Punkten ist hinreichend, aber nicht zwingend notwendig, um die Prüfung zu bestehen.

Viel Erfolg!

Die Klasse Raum (10 Punkte)

Objekte dieser Klasse repräsentieren einen Raum, in dem eine Klausur geschrieben werden kann. Ein Raum hat einen Namen und eine Kapazität, die angibt, wieviele Studenten maximal in dem Raum eine Klausur schreiben können.

Implementieren Sie die folgenden Methoden:

- `public Raum(String name, int capacity)`
Dieser Konstruktor soll einen neuen Raum mit den gegebenen Parametern erzeugen.
Ist `name` `null`, soll eine `NullPointerException` geworfen werden.
Ist `name` leer, soll eine `IllegalArgumentException` geworfen werden.
Ist `capacity` kleiner oder gleich 0, soll eine `IllegalArgumentException` geworfen werden.
- `public String getName()`
Gibt den Namen des Raums zurück.
- `public int getCapacity()`
Gibt die Kapazität des Raums zurück.
- `public boolean equals(Object o)`
Überschreiben Sie diese Methode so, dass zwei Räume gleich sind, wenn ihre Namen gleich sind.
- `public int hashCode()`
Überschreiben Sie diese Methode so, dass sie den API-Vertrag zwischen `equals` und `hashCode` einhält.
- `public String toString()`
Überschreiben Sie diese Methode so, dass sie den Namen des Raums zurückgibt.

Die Klasse Pruefung (18 Punkte)

Objekte dieser Klasse repräsentieren eine Prüfung. Eine Prüfung speichert ihren Namen und die Anzahl der Teilnehmer. Bei Erstellung einer Prüfung existieren noch keine Noten, es ist aber möglich später Noten für die Teilnehmer hinzuzufügen.

Implementieren Sie die folgenden Methoden:

- `public Pruefung(String name, int numParticipants)`
Dieser Konstruktor soll eine neue Prüfung mit den gegebenen Parametern erzeugen.
Ist `name` `null`, soll eine `NullPointerException` geworfen werden.
Ist `name` leer, soll eine `IllegalArgumentException` geworfen werden.
Ist `numParticipants` kleiner oder gleich 0, soll eine `IllegalArgumentException` geworfen werden.
- `public String getName()`
Gibt den Namen der Prüfung zurück.
- `public int getNumParticipants()`
Gibt die Anzahl der Teilnehmer zurück.
- `public void setGrades(Map<Student, Integer> grades)`
Speichert die übergebenen Noten ab.
Wurden die Noten bereits zuvor gesetzt, soll stattdessen eine `IllegalStateException` geworfen werden.
Ist `grades` `null` soll eine `NullPointerException` geworfen werden.
Ist die Größe von `grades` ungleich der Anzahl der Prüfungsteilnehmer, soll eine `IllegalArgumentException` geworfen werden.

Anmerkung:

Die Klasse `Student` wird von Ihnen im Anschluss implementiert. Sie benötigen keine Methoden der Klassen, sondern verwenden nur `Student`-Objekte als Schlüssel in der `Noten-Map`.

- `public Optional<Map<Student, Integer>> getGrades()`
Gibt die gesetzten Noten, gepackt in ein `Optional`, zurück.
Falls noch keine Noten gesetzt wurden, soll stattdessen ein leeres `Optional` zurückgegeben werden.
- `public Optional<Integer> getGrade(Student s)`
Gibt die Note des Studenten, gepackt in ein `Optional`, zurück.
Falls noch keine Noten gesetzt wurden oder der Student nicht in den Noten vorhanden ist, soll stattdessen ein leeres `Optional` zurückgegeben werden.
- `public boolean equals(Object o)`
Überschreiben Sie diese Methode so, dass zwei Prüfungen gleich sind, wenn ihre Namen gleich sind.
- `public int hashCode()`
Überschreiben Sie diese Methode so, dass sie den API-Vertrag zwischen `equals` und `hashCode` einhält.
- `public String toString()`
Überschreiben Sie diese Methode so, dass sie den Namen der Prüfung zurückgibt.

Die Klasse `Student` (15 Punkte)

Objekte dieser Klasse repräsentieren Studenten, die an Prüfungen teilnehmen können. Sie sind bereits in der `Pruefung`-Klasse mit dieser Klasse in Kontakt gekommen. Studenten speichern ihren Namen sowie die Prüfungen, zu denen sie angemeldet sind.

Implementieren Sie die folgenden Methoden:

- `public Student(String name)`
Dieser Konstruktor erstellt einen neuen Studenten mit gegebenem Namen.
Ist `name` `null`, soll eine `NullPointerException` geworfen werden.
Ist `name` leer, soll eine `IllegalArgumentException` geworfen werden.
Ein Student soll zu Beginn zu keiner Prüfung angemeldet sein.
- `public String getName()`
Gibt den Namen des Studenten zurück.
- `public void addExam(Pruefung p)`
Speichert ab, dass der Student zur gegebenen Prüfung angemeldet ist.
Falls `p` `null` ist, soll stattdessen eine `NullPointerException` geworfen werden.
Falls `p` bereits zuvor hinzugefügt worden ist, soll eine `IllegalArgumentException` geworfen werden.
Die Reihenfolge, in der Prüfungen hinzugefügt werden ist wichtig, also wählen Sie eine geeignete Datenstruktur zum Abspeichern.
- `public List<Pruefung> getExams()`
Gibt alle hinzugefügten Prüfungen zurück.
- `public List<Optional<Integer>> getGrades()`
Gibt die Noten der angemeldeten Prüfungen zurück. An Index `i` soll die Note stehen, die zur Prüfung gehört, die als `i`-tes hinzugefügt wurde.
Hat eine Prüfung noch keine Noten, soll ein leeres `Optional` in der Liste stehen. Ansonsten die Note, die der Student in der Prüfung hat, gepackt in einen `Optional`.
- `public String toString()`
Überschreiben Sie diese Methode so, dass sie den Namen des Studenten zurückgibt.

`equals` und `hashCode` müssen nicht überschrieben werden. Jedes `Student`-Objekt ist also nur genau zu sich selbst gleich.

Die Klasse `Analyse` (47 Punkte)

Die Aufgabe dieser Klasse ist es aus einer bestehenden Klausurplanung Informationen abzuleiten. Eine Klausur besteht aus zwei Maps:

- `Zeitplan ( Map<Pruefung, String> )`:
Speichert ab, zu welchem Zeitpunkt eine Prüfung stattfindet.
Der Zeitpunkt wird hier als `String` abgespeichert.
- `Raumplan ( Map<Pruefung, Raum> )`:
Speichert ab, in welchem Raum eine Prüfung stattfindet.

Implementieren Sie die folgenden Methoden:

- `public Analyse(Map<Pruefung, String> timeSchedule, Map<Pruefung, Raum> roomSchedule)`
Dieser Konstruktor erstellt ein neues `Analyse`-Objekt mit gegebenen Parametern.
Ist einer der Parameter `null`, soll eine `NullPointerException` geworfen werden.
Falls die Maps nicht genau die gleichen Prüfungen enthalten (also falls die `keySet()`s nicht gleich sind), soll eine `IllegalArgumentException` geworfen werden.
Sie können allerdings annehmen, dass nicht mehrere Prüfungen für die selbe Zeit im selben Raum eingeplant sind.

Bei den folgenden Methoden können Sie annehmen, dass die übergebenen Parameter sinnvoll sind und nicht auf `null` oder ähnliches überprüft werden müssen.

- `public Set<String> getAllTimes()`
Geben Sie alle Klausurzeitpunkte zurück, die in `timeSchedule` vorkommen.
- `public Set<Raum> getAllRooms()`
Geben Sie alle Räume zurück, die in `roomSchedule` vorkommen.
- `public Set<Pruefung> getAllExams()`
Geben Sie alle geplanten Prüfungen zurück.
Erinnerung:
Im Konstruktor haben Sie überprüft, dass `timeSchedule` und `roomSchedule` die selben Prüfungen enthalten.

- `public Set<Pruefung> getExamsAtTime(String time)`
Geben Sie zurück, welche Prüfungen zu gegebenen Zeitpunkt stattfinden.
Hinweis:
Mit `.entrySet()` können Sie sich alle Einträge einer Map als Set liefern zu lassen. Dieses Entry-Set enthält `Entry`-Objekte, auf denen `.getKey()` und `.getValue()` aufgerufen werden können.
Das Entry-Set bietet sich an, um mit einer Schleife hindurch zu iterieren oder in einem `Stream` zu verarbeiten.

- `public int getExamCountAtTime(String time)`
Geben Sie zurück, wieviele Prüfungen zu gegebenen Zeitpunkt stattfinden.
- `public Set<Pruefung> getExamsInRoom(Raum room)`
Geben Sie zurück, welche Prüfungen in dem gegebenen Raum stattfinden.
- `public int getExamCountInRoom(Raum room)`
Geben Sie zurück, wieviele Prüfungen in dem gegebenen Raum stattfinden.
- `public Optional<Pruefung> getExamAt(String time, Raum room)`
Geben Sie die Prüfung zurück, die zur Zeit `time` im Raum `room` eingeplant ist.
Falls keine solche Klausur existiert, soll ein leeres `Optional` zurückgegeben werden.

Anmerkung:

Nach der Annahme, dass es keine Doppelbelegung (gleiche Zeit, gleicher Ort, mehrere Prüfungen gibt), kann es nicht mehr als eine passende Prüfung geben.

- `public Optional<Integer> getBestGradeOfStudent(Student s)`

Geben Sie die beste Note des Studenten zurück.

Die Noten sind hier als Schulnoten zu verstehen, insbesondere bedeutet ein geringerer numerischer Wert eine bessere Note.

Falls der Student noch keine Klausur mit gesetzten Noten hat, soll stattdessen ein leeres `Optional` zurückgegeben werden.

Hinweis:

Verwenden Sie hier **nicht** `timeSchedule` und `roomSchedule` oder sonstige Methoden dieser Klasse, die von diesen abhängen.

- `public Set<Pruefung> getBestExamsOfStudent(Student s)`

Geben Sie alle Prüfungen zurück, in denen der Student seine Bestnote erreicht hat.

Gibt es noch keine Bestnote, soll hier ein leeres `Set` zurückgegeben werden.

Hinweis:

Verwenden Sie hier **nicht** `timeSchedule` und `roomSchedule` oder sonstige Methoden dieser Klasse, die von diesen abhängen.

- `public boolean hasCollision(Student s)`

Geben Sie zurück, ob bei dem Studenten eine Kollision auftritt, also er zu mehreren Klausuren angemeldet ist, die zum selben Zeitpunkt stattfinden.

- `public Set<Student> getCollisionStudents(Set<Student> students)`

Prüft für die gegebenen Studenten, ob eine Kollision auftritt und gibt die betroffenen Studenten zurück.

Ende